

EOAT Command Center Architecture Improvement Plan

Purpose

This document lays out a detailed restructuring and improvement plan for the EOAT Command Center app. The goal is to make the app easier to expand, easier to test, safer to maintain, and less likely to turn into a cursed spreadsheet-powered swamp monster every time a new audit field gets added.

This plan intentionally focuses on **overall structure, maintainability, correctness, and feature scalability**. It does **not** focus on raw latency/performance tuning, because that is already being handled separately.

This document also intentionally excludes tag-category changes for now.

Executive Summary

The app already has a strong foundation:

- It is local-first.
- It uses a real desktop dashboard.
- It keeps real plant data outside the GitHub repo.
- It has a project-root-based data model.
- It has a workbook-driven workflow that matches the real internship deliverables.
- It has pages for audits, reports, workbook health, scheduled reports, notes, photos, progress, pilot candidates, FMEA, KPI tracking, and handoff.
- It already has some good architectural patterns, especially the dashboard page registry.

However, the EOAT Audit workflow is becoming the central complexity hotspot.

Right now, adding or changing an audit field can require changes across many separate areas:

- UI section layout
- UI grouping
- widget creation
- dropdown values
- workbook schema
- workbook migration
- audit normalization
- validation
- audit coach progress logic
- field visibility rules

- compatibility logic
- reports
- tests

That is too many places. It works now, but it makes future feature additions risky. Every new audit field becomes a scavenger hunt through the codebase, except instead of treasure there is an off-by-one Excel column bug wearing a little hat.

The biggest recommendation is to move toward a **declarative audit schema** where each field is defined once and then reused by the UI, workbook layer, validation, progress engine, audit coach, reports, and migrations.

The core improvement themes are:

1. Create a central audit field registry.
2. Extract audit completion/progress rules into a dedicated completion policy engine.
3. Split the large Audit page into smaller controllers.
4. Add explicit support for optional field groups such as Cylinders.
5. Add a truthful manual completion override system.
6. Separate workbook persistence from audit domain logic.
7. Add versioned workbook migrations.
8. Add a shared audit relationship service for Press View and compatibility logic.
9. Improve dashboard routing and search result handling.
10. Harden the event bus.
11. Add stable internal field IDs separate from user-facing labels.
12. Strengthen testing, linting, and release discipline.

Guiding Design Principles

1. Define audit fields once

A field such as `# of Cylinders` should not require separate hardcoded updates in five different places. The app should have one authoritative field definition that includes:

- Internal field ID
- User-facing label
- Workbook header
- UI section
- UI group
- widget type
- dropdown options
- default value
- validation rules
- visibility behavior
- completion/progress behavior

- workbook storage target
- legacy header aliases

The rest of the app should derive behavior from that source.

2. Keep the UI thin

The UI should display and coordinate the workflow. It should not be responsible for knowing every rule about audit completion, workbook normalization, legacy field migration, field applicability, and compatibility relationships.

A page class should not also be a workbook migration engine, progress calculator, draft manager, annotation router, lookup controller, and compatibility wizard. That is how software starts developing folklore.

3. Make progress truthful

The audit coach should not simply say “100% complete” because a status field says complete. It should distinguish between:

- Verified complete
- Complete by manual override
- Missing but intentionally ignored
- Not applicable
- Unknown / not checked
- Needs follow-up
- Hidden stale value conflict

This matters because the app is being used to support real manufacturing engineering work. A fake 100% is worse than an honest 84%.

4. Separate physical audits from compatibility-derived rows

Physical audit rows and compatibility rows should remain distinct. Compatibility rows should not pollute physical audit completion logic, and physical audit data should not be miscounted when viewing compatibility status.

5. Prefer additive migrations over destructive changes

Workbook changes should be safe, versioned, reversible when possible, and backed up before modifications.

Do not silently delete or overwrite existing data.

6. Keep real data outside the repo

The app should continue enforcing the existing local-first and data-safety boundary. Source code and sanitized demo data belong in GitHub. Real workbooks, photos, logs, reports, capacity files, and plant-specific data do not.

Feature / Refactor 1: Central Audit Field Registry

Problem

Audit field definitions are currently scattered across several conceptual locations:

- UI sections and tab layout
- UI group layout
- widget creation logic
- dropdown values
- workbook schema JSON
- field applicability rules
- audit coach field classification
- validation rules
- default rules
- workbook column ordering
- migration/legacy repair logic

This makes changes expensive and risky. A field can appear correctly in the UI but fail to save correctly. Or it can save correctly but not count correctly in the audit coach. Or it can appear in the workbook but not participate in validation. Naturally, because apparently one field needs seven passports to cross the app border.

Goal

Create one central schema/registry that defines every audit field and exposes helper functions that other parts of the app can use.

Proposed File

```
core/audit/schema.py
```

Proposed Data Model

```
from dataclasses import dataclass
from typing import Literal
```

```

WidgetType = Literal[
    "line_edit",
    "text_edit",
    "combo",
    "checkbox",
]

StorageTarget = Literal[
    "eoa_master",
    "robot_info",
    "audit_metadata",
    "none",
]

ProgressPolicy = Literal[
    "normal",
    "excluded",
    "optional_group_trigger",
    "optional_group_dependent",
    "manual_override_metadata",
]

VisibilityRule = Literal[
    "always",
    "vacuum_tooling",
    "gripper_tooling",
    "sensor_detail",
    "electrical_detail",
    "quick_disconnect_detail",
    "compatible_row_hidden",
]

@dataclass(frozen=True)
class AuditFieldSpec:
    field_id: str
    label: str
    workbook_header: str
    section: str
    group: str
    widget_type: WidgetType
    storage_target: StorageTarget = "eoa_master"
    dropdown_values: tuple[str, ...] = ()
    editable_dropdown: bool = False
    include_blank: bool = True
    default: str | None = None
    numeric: bool = False
    required_for_audited: bool = False

```

```

required_for_compatible: bool = False
important: bool = False
visibility_rule: VisibilityRule = "always"
progress_policy: ProgressPolicy = "normal"
optional_group: str | None = None
legacy_headers: tuple[str, ...] = ()
tags: tuple[str, ...] = ()
help_text: str = ""

```

Example Field Definitions

```

AUDIT_FIELDS = (
    AuditFieldSpec(
        field_id="audit_id",
        label="Audit ID",
        workbook_header="Audit ID",
        section="Audit Header",
        group="Audit Identity",
        widget_type="line_edit",
        required_for_audited=True,
        progress_policy="normal",
    ),
    AuditFieldSpec(
        field_id="eoat_type",
        label="EOAT Type",
        workbook_header="EOAT Type",
        section="EOAT Type and Tooling",
        group="EOAT Classification",
        widget_type="combo",
        dropdown_values=("Vacuum", "Mechanical / Gripper", "Hybrid", "Unknown /
Needs Review", "Miscellaneous"),
        required_for_audited=True,
        visibility_rule="always",
        progress_policy="normal",
    ),
    AuditFieldSpec(
        field_id="cylinder_count",
        label="# of Cylinders",
        workbook_header="# of Cylinders",
        section="EOAT Type and Tooling",
        group="Cylinder Details",
        widget_type="line_edit",
        numeric=True,
        visibility_rule="always",
        progress_policy="optional_group_trigger",
        optional_group="cylinders",
    ),
)

```

```

    ),
    AuditFieldSpec(
        field_id="cylinder_type",
        label="Cylinder Type",
        workbook_header="Cylinder Type",
        section="EOAT Type and Tooling",
        group="Cylinder Details",
        widget_type="combo",
        dropdown_values=("Linear", "Rotary"),
        default="Linear",
        visibility_rule="always",
        progress_policy="optional_group_dependent",
        optional_group="cylinders",
    ),
)

```

Helper Functions

```

def all_audit_fields() -> tuple[AuditFieldSpec, ...]:
    ...

def field_by_id(field_id: str) -> AuditFieldSpec:
    ...

def field_by_header(header: str) -> AuditFieldSpec | None:
    ...

def fields_for_section(section: str) -> list[AuditFieldSpec]:
    ...

def fields_grouped_by_section() -> dict[str, dict[str, list[AuditFieldSpec]]]:
    ...

def expected_workbook_headers(storage_target: str = "eoa_master") -> list[str]:
    ...

def dropdown_values_for(field_id_or_header: str) -> tuple[str, ...]:
    ...

```

Modules That Should Use the Registry

The following code should gradually stop hardcoding field lists and instead consume the registry:

- Audit page UI builder
- Widget factory
- Field visibility rules
- Completion/progress calculator
- Audit coach
- Workbook schema generation
- Workbook validation
- Workbook migrations
- Audit normalization
- Audit validation
- Reports
- Search indexing
- Compatibility logic

Acceptance Criteria

- All existing audit fields are represented in the registry.
- UI sections can be generated from the registry.
- Workbook expected headers can be generated from the registry.
- Dropdowns can be generated from the registry.
- Numeric validation can be generated from the registry.
- Existing tests still pass.
- No user-facing behavior changes in the first phase.
- The old field constants can remain temporarily as wrappers/aliases.

Testing Requirements

Add tests for:

1. Registry contains all current workbook headers.
 2. Registry has no duplicate `field_id` values.
 3. Registry has no duplicate workbook headers for the same storage target unless explicitly allowed.
 4. UI section generation from registry matches the expected current section order.
 5. Dropdown fields expose expected values.
 6. Numeric fields are correctly marked numeric.
 7. Legacy header aliases resolve correctly.
-

Feature / Refactor 2: Dedicated Audit Completion Policy Engine

Problem

The audit coach currently classifies fields mainly by whether they apply and whether they have a meaningful value. This works for simple forms, but the audit workflow now needs more nuanced rules.

Examples:

- Some notes fields should not count against completion.
- Some default values should not count as intentional user input.
- Cylinder fields should be visible but optional unless used.
- Manual completion override should make a specific audit 100% complete without pretending every field was filled.
- Compatibility rows should not behave like physical audit rows.
- Unknown / Not Checked should be explicit but not verified complete.

If these rules are implemented as one-off exceptions inside the UI or coach, the logic will become brittle.

Goal

Create a dedicated completion engine that decides which fields count, which fields are complete, which fields are ignored, and why.

Proposed File

```
core/audit/completion.py
```

Proposed Data Model

```
from dataclasses import dataclass
from typing import Literal

CompletionState = Literal[
    "verified_complete",
    "missing",
    "unknown_not_checked",
    "not_applicable",
    "follow_up_needed",
    "stale_conflict",
    "excluded",
```

```

        "ignored_by_optional_group",
        "ignored_by_manual_override",
    ]

    @dataclass(frozen=True)
    class FieldCompletionStatus:
        field_id: str
        label: str
        workbook_header: str
        section: str
        group: str
        state: CompletionState
        value: str
        applies: bool
        counts_toward_progress: bool
        required: bool = False
        important: bool = False
        reason: str = ""

    @dataclass(frozen=True)
    class SectionCompletionStatus:
        section: str
        applicable_count: int
        counted_count: int
        verified_count: int
        missing_count: int
        ignored_count: int
        percent_complete: int
        fields: tuple[FieldCompletionStatus, ...]

    @dataclass(frozen=True)
    class AuditCompletionSummary:
        audit_id: str
        percent_complete: int
        complete_by_manual_override: bool
        verified_count: int
        counted_count: int
        missing_fields: tuple[str, ...]
        ignored_fields: tuple[str, ...]
        sections: tuple[SectionCompletionStatus, ...]
        explanation: str

```

Completion Rules

Normal fields

A normal field counts toward progress if:

- The field applies to the current audit row.
- The field is not excluded from completion.
- The field is not ignored by optional group logic.
- The field is not ignored by manual override.

It is complete if it has a meaningful value.

Excluded fields

Fields with `progress_policy = "excluded"` should never reduce completion.

Examples:

- `Notes`
- `Tubing Routing Notes`
- Other purely optional comment fields as needed

These fields can still save, load, display, search, and export. They simply do not count against audit completion.

Unknown / Not Checked

`Unknown / Not Checked` should be treated as an explicit audit state, but not as verified complete.

This means:

- It should not look blank.
- It should not be marked as verified complete.
- It should remain visible to the audit coach as something that may need follow-up.

Not applicable

A field that does not apply should not count against completion.

If it contains a meaningful value while not applicable, it should be flagged as a stale conflict.

Optional groups

An optional group should only count if the user actually uses it.

For example, the Cylinder group should be ignored when unused.

Manual override

If an audit has a manual completion override:

- The audit-level percentage should be 100%.
- The system should not pretend every field is verified.
- Missing fields should be classified as ignored by manual override.
- The UI should show a clear explanation.

Example explanation:

This audit is marked complete by manual override. Blank fields are ignored for completion percentage, but the audit was not fully field-verified.

Acceptance Criteria

- Completion logic can be tested without launching the UI.
- Audit coach uses completion results instead of duplicating completion rules.
- Optional fields can be excluded declaratively.
- Optional groups can be ignored unless triggered.
- Manual override is represented truthfully.
- Existing completion behavior remains the same unless specifically changed by new rules.

Testing Requirements

Add tests for:

1. A fully filled audit calculates 100%.
 2. A blank required field reduces completion.
 3. `Unknown / Not Checked` does not count as verified complete.
 4. `N/A` fields do not count when not applicable.
 5. Meaningful values in non-applicable fields create stale conflict statuses.
 6. Excluded fields do not reduce completion.
 7. Empty optional groups do not reduce completion.
 8. Triggered optional groups do participate in completion.
 9. Manual override makes audit-level percent 100%.
 10. Manual override still preserves field-level truth.
-

Feature 3: Cylinder Mini-Section

Purpose

Add cylinder tracking to the EOAT Audit form. Some EOATs use cylinders in addition to or instead of cups/grippers, so cylinders need to be captured as their own tooling detail.

This should be done through the new field registry and completion policy system where possible.

Required UI Location

Add a new mini-section on the `EOAT Type and Tooling` tab.

Recommended group name:

Cylinder Details

Required Fields

of Cylinders
Cylinder Type

Field Behavior

`# of Cylinders`

- Widget type: line edit
- Numeric only
- Accept whole numbers only
- Minimum: 0
- Recommended maximum: 9999, matching other count fields
- Empty is allowed

`Cylinder Type`

- Widget type: dropdown
- Values:
 - Linear
 - Rotary
- Default:
 - Linear
- Editable:
 - No, unless there is a strong reason to allow custom types later

Visibility Rules

The Cylinder section should **always be visible**.

It should not disappear for:

- Vacuum EOAT
- Mechanical / Gripper EOAT
- Hybrid EOAT
- Miscellaneous EOAT
- Unknown / Needs Review EOAT
- Blank EOAT Type

The reason is simple: cylinders are a possible EOAT component across multiple mechanical configurations, and hiding them based on EOAT Type would create false assumptions.

Completion / Progress Rules

The Cylinder section should be optional unless used.

Unused Cylinder Section

If `# of Cylinders` is empty and `Cylinder Type` is only sitting at its untouched default of `Linear`, then the cylinder group should not count against audit completion.

This prevents every audit from accidentally appearing to have cylinder data just because the dropdown has a default value.

Used Cylinder Section

If `# of Cylinders` has a value, then the cylinder section is considered intentionally used.

At that point:

- `# of Cylinders` should count toward completion.
- `Cylinder Type` should count toward completion.
- `Cylinder Type` should default to `Linear` if blank.

Optional Advanced Rule

If the app adds field touch tracking, then changing `Cylinder Type` manually could also trigger the optional group even if `# of Cylinders` is blank.

However, the simpler and safer first version should use this rule:

Cylinder group is active only when # of Cylinders is non-empty.

That is easier to test and avoids ghost data.

Workbook Behavior

Add these workbook headers to `EOAT Inventory`:

of Cylinders
Cylinder Type

Recommended placement:

Place them in the tooling column group, after gripper fields and before cup fields, or after cup fields and before physical details. The cleanest UI grouping is probably:

EOAT Type
EOAT Moves
Connection Type
Number of Parts Picked
of Grippers
Gripper Type
Gripper Model
Gripper Size
of Cylinders
Cylinder Type
of Cups
Cup Type/Material
Cup Diameter/Size
Vacuum Generator Type
Estimated EOAT Weight

This keeps all end-effector component counts together.

Schema Additions

Add to `data_templates/workbook_schema.json` under `EOAT Inventory.required_headers`:

```
"# of Cylinders",  
"Cylinder Type"
```

Also add the fields to the central registry.

Validation Rules

- # of Cylinders must be blank or a non-negative integer.
- Cylinder Type must be either blank, Linear, or Rotary.
- If # of Cylinders is filled and Cylinder Type is blank, default to Linear during normalization.
- If # of Cylinders is blank, do not force Cylinder Type into the workbook as meaningful data unless the app intentionally stores defaults.

Normalization Rules

Recommended normalized save behavior:

```
If # of Cylinders is blank:
    save # of Cylinders as blank or N/A based on existing workbook convention
    do not treat Cylinder Type = Linear as meaningful audit completion

If # of Cylinders is filled:
    save # of Cylinders as entered
    save Cylinder Type as selected, defaulting to Linear if blank
```

Be careful with automatic N/A filling. If the existing app converts blanks to N/A, the completion engine must still know that an unused cylinder group should not reduce completion.

UI Testing

Add tests for:

1. Cylinder group appears on the EOAT Type and Tooling tab.
2. Cylinder group remains visible for Vacuum EOAT.
3. Cylinder group remains visible for Mechanical / Gripper EOAT.
4. Cylinder group remains visible for Hybrid EOAT.
5. Cylinder group remains visible for Miscellaneous EOAT.
6. # of Cylinders accepts numeric values.
7. # of Cylinders rejects non-numeric values.
8. Cylinder Type dropdown contains Linear and Rotary.
9. Cylinder Type defaults to Linear.

Completion Testing

Add tests for:

1. Blank cylinder count does not reduce completion.
2. Blank cylinder count plus default Linear does not reduce completion.
3. Cylinder count filled with 1 activates the cylinder group.

4. Activated cylinder group counts both cylinder fields.
 5. Activated cylinder group defaults missing cylinder type to `Linear`.
 6. Existing audits without cylinder fields still load successfully.
-

Feature 4: Manual Completion Override

Purpose

Allow the user to manually mark a specific audit as complete even when blank or unused fields remain.

This is needed because not every audit needs every visible field filled out. Sometimes a form is legitimately complete for the real-world situation even if the generic template contains fields that do not apply cleanly.

The key requirement is truthfulness:

The app should not pretend every field was verified. It should state that the audit was marked complete by manual override.

UI Location

Add a button on the EOAT Audit page near the existing save controls.

Suggested label:

Manual Override: Mark Audit Complete

This button should be visually distinct from normal save actions.

Recommended placement:

- Same lower button row as `Save Audit Entry`, `Save Draft`, and `Clear Form`
- Or inside the Audit Coach panel as an advanced action

Best recommendation:

Put it near the Audit Coach panel because it directly affects audit completion status.

Confirmation Popup

When clicked, show a confirmation popup before applying the override.

Popup Title

Manual Completion Override

Popup Message

This will manually mark this audit as complete even though some fields may still be blank. Blank fields currently remaining on this audit will be ignored by the audit coach for completion percentage purposes.

This only affects the current audit. It does not change global validation rules.

Are you sure you want to continue?

Buttons

Cancel
Mark Complete

Default/focused action should be `Cancel`.

Humans click buttons like raccoons in a vending machine, so the safe default matters.

Data Model

Add audit-specific metadata fields.

Minimum workbook columns:

Manual Completion Override
Manual Completion Override Timestamp
Ignored Empty Fields At Override

Optional additional columns:

Manual Completion Override User
Manual Completion Override Note
Manual Completion Override Version

Recommended field definitions:

```

AuditFieldSpec(
    field_id="manual_completion_override",
    label="Manual Completion Override",
    workbook_header="Manual Completion Override",
    section="Audit Metadata",
    group="Completion Override",
    widget_type="line_edit",
    storage_target="eoat_master",
    progress_policy="manual_override_metadata",
)

AuditFieldSpec(
    field_id="manual_completion_override_timestamp",
    label="Manual Completion Override Timestamp",
    workbook_header="Manual Completion Override Timestamp",
    section="Audit Metadata",
    group="Completion Override",
    widget_type="line_edit",
    storage_target="eoat_master",
    progress_policy="manual_override_metadata",
)

AuditFieldSpec(
    field_id="ignored_empty_fields_at_override",
    label="Ignored Empty Fields At Override",
    workbook_header="Ignored Empty Fields At Override",
    section="Audit Metadata",
    group="Completion Override",
    widget_type="text_edit",
    storage_target="eoat_master",
    progress_policy="manual_override_metadata",
)

```

These metadata fields do not need to be visible as normal audit fields. They can be shown in a status badge or details panel instead.

Behavior

When user confirms override:

1. Read current form values.
2. Calculate current missing/blank fields.
3. Store override metadata.
4. Save the audit.
5. Refresh the audit coach.
6. Show completion as 100% by manual override.

7. Display a visible note/badge.

Example status badge:

Manual completion override applied. This audit is treated as 100% complete, but some fields were not field-verified.

Important Behavior Rules

Applies only to current audit

The override must not change global required fields, global progress behavior, or any other audit.

Does not delete data

Existing form values must remain unchanged.

Does not fake verification

The app should not mark all blank fields as verified. It should mark the audit as complete by override.

Should persist after reload

If the user reloads the audit later, the override should remain visible and the audit coach should still show 100% by override.

Should not silently disappear after edits

If the user edits an overridden audit, the override should remain active unless a deliberate clear/remove override feature is implemented.

Should save cleanly

After applying the override, the user should not immediately get an unsaved-edits popup if the app successfully saved the override.

Optional Clear Override Feature

A later improvement could add:

Clear Manual Completion Override

This should require confirmation and should restore normal completion calculation.

This is optional and should not block initial implementation.

Testing Requirements

Add tests for:

1. Manual override button is visible.
 2. Clicking the button opens confirmation popup.
 3. Cancel does not apply override.
 4. Confirm applies override to current audit.
 5. Override stores metadata.
 6. Override saves to workbook.
 7. Reloading the audit preserves override.
 8. Audit coach shows 100% completion by override.
 9. Field-level statuses remain truthful.
 10. Other audits are unaffected.
 11. Save behavior remains clean after override.
 12. Unsaved change popup does not appear incorrectly after successful override save.
-

Feature / Refactor 5: Split the Audit Page into Smaller Controllers

Problem

The Audit page currently has too many responsibilities. It handles:

- UI layout
- field widget creation
- form value get/set
- dirty state tracking
- audit ID generation
- audit loading
- audit duplication
- audit saving
- draft recovery
- unsaved-change confirmation
- audit coach refresh
- field visibility
- machine lookup
- capacity match display
- compatibility tooling
- annotation target creation
- field tag dialogs

- follow-up creation
- result panel messages

This makes the file harder to understand and makes every new feature more dangerous.

Goal

Turn the Audit page into a thin shell that delegates work to focused controllers.

Proposed Structure

```
app/pages/audit/  
├── __init__.py  
├── page.py  
├── form_builder.py  
├── field_widgets.py  
├── form_state.py  
├── visibility_controller.py  
├── defaults_controller.py  
├── load_controller.py  
├── save_controller.py  
├── draft_controller.py  
├── coach_controller.py  
├── annotation_controller.py  
├── machine_lookup_controller.py  
├── compatibility_controller.py  
└── result_messages.py
```

Proposed Responsibilities

`page.py`

Contains the main `AuditPage(QWidget)` class.

Responsibilities:

- Build high-level layout
- Instantiate controllers
- Connect major UI signals
- Provide page lifecycle hooks
- Coordinate controller interactions

Should not contain detailed business rules.

`form_builder.py`

Builds audit section tabs and groups from the audit field registry.

Responsibilities:

- Create section tabs
- Create group boxes
- Add field rows
- Attach labels
- Attach tag buttons if needed
- Return a structured form object

`field_widgets.py`

Creates widgets from `AuditFieldSpec`.

Responsibilities:

- Create line edits
- Create numeric line edits
- Create text edits
- Create dropdowns
- Apply validators
- Attach field-specific signal hooks

`form_state.py`

Handles form values and dirty tracking.

Responsibilities:

- Get current form values
- Set form values
- Mark baseline
- Compare current values to baseline
- Ignore generated IDs when appropriate
- Ignore lookup-only changes when appropriate

`visibility_controller.py`

Applies field visibility and section/group visibility rules.

Responsibilities:

- Determine whether a field should show
- Hide/show field rows

- Hide/show group boxes
- Explain non-applicable fields
- Clear hidden values when saving if required

`defaults_controller.py`

Centralizes default behavior.

Responsibilities:

- Initial form defaults
- Part-present sensor defaults
- Quick disconnect defaults
- Changeover difficulty defaults
- Cylinder Type default behavior

Existing defaults logic can be moved here gradually.

`load_controller.py`

Handles audit loading and duplication.

Responsibilities:

- Load existing audit
- Populate form
- Handle Empty Only mode setup
- Duplicate audits safely
- Generate new audit IDs
- Refresh audit selector

`save_controller.py`

Handles audit save workflow.

Responsibilities:

- Collect form values
- Normalize values
- Validate values
- Write to workbook
- Save robot info fields
- Trigger compatibility update prompt if needed
- Apply manual completion override metadata
- Emit audit saved event

`draft_controller.py`

Handles drafts.

Responsibilities:

- Save draft
- Restore draft
- Discard draft
- Offer draft recovery
- Coordinate unsaved-change confirmation

`coach_controller.py`

Coordinates with the audit coach.

Responsibilities:

- Refresh coach summary
- Open field from coach
- Mark field unknown
- Create follow-up from coach
- Apply manual completion override

`annotation_controller.py`

Handles field-level annotations.

Responsibilities:

- Create/get annotation targets
- Open field tag dialog
- Refresh tag indicators
- Navigate to tagged field

`machine_lookup_controller.py`

Handles press/machine lookup.

Responsibilities:

- Lookup machine info
- Populate robot/part fields
- Show capacity matches
- Resolve lookup conflicts

`compatibility_controller.py`

Handles compatibility interactions from the audit page.

Responsibilities:

- Build compatibility candidates
- Preview compatibility impacts
- Create/update compatibility rows
- Keep compatibility side effects away from basic save logic

Migration Strategy

Do not split everything in one pass.

Recommended sequence:

1. Extract `form_state.py` first.
2. Extract `field_widgets.py` second.
3. Extract `form_builder.py` third.
4. Extract `draft_controller.py` fourth.
5. Extract `visibility_controller.py` fifth.
6. Extract `coach_controller.py` sixth.
7. Extract annotation and lookup controllers last.

Each extraction should preserve behavior and pass tests before continuing.

Acceptance Criteria

- `AuditPage` gets substantially smaller.
- Existing audit behavior remains unchanged.
- New field additions require fewer edits.
- Unit tests can target form state, completion, and visibility without launching the full app.
- UI smoke tests still pass.

Feature / Refactor 6: Separate Workbook Persistence from Audit Domain Logic

Problem

Audit domain logic and workbook I/O are currently tightly connected.

The app needs to know:

- What an audit field means
- Whether a field applies
- Whether a field is valid
- How to normalize values
- How to migrate workbook headers
- How to insert workbook columns
- How to style workbook columns
- How to write rows
- How to repair legacy rows
- How to back up files
- How to detect workbook locks

These are related, but they are not the same responsibility.

Goal

Separate audit domain logic from workbook persistence logic.

Proposed Structure

```
core/audit/  
├─ schema.py  
├─ models.py  
├─ normalizer.py  
├─ validator.py  
├─ completion.py  
├─ applicability.py  
├─ repository.py  
├─ workbook_repository.py  
├─ workbook_migrations.py  
├─ legacy_repair.py  
└─ save_service.py
```

Proposed Modules

`models.py`

Defines audit domain objects.

```
@dataclass(frozen=True)  
class AuditEntry:  
    audit_id: str
```

```
values: dict[str, str]
entry_type: str = "Audited"
```

normalizer.py

Handles data normalization independent of Excel.

Responsibilities:

- Apply defaults
- Clear non-applicable fields
- Normalize gripper model values
- Normalize blank values
- Apply part-present sensor defaults
- Apply cylinder defaults if used

validator.py

Handles validation independent of Excel.

Responsibilities:

- Required field validation
- Numeric field validation
- Dropdown value validation
- Semantic warnings
- Hybrid completeness warnings
- Cylinder validation

repository.py

Defines an abstract repository interface.

```
class AuditRepository(Protocol):
    def list_audits(self) -> list[AuditEntry]: ...
    def get_audit(self, audit_id: str) -> AuditEntry | None: ...
    def save_audit(self, entry: AuditEntry) -> SaveResult: ...
```

workbook_repository.py

Implements the repository using Excel workbooks.

Responsibilities:

- Read rows from workbook

- Write rows to workbook
- Locate rows by Audit ID
- Create missing sheets
- Ensure headers exist
- Preserve workbook formatting
- Back up before modifications
- Detect workbook locks

`workbook_migrations.py`

Runs workbook schema upgrades.

Responsibilities:

- Add missing columns
- Rename legacy columns
- Move columns into expected order
- Apply data validation
- Record migration history

`legacy_repair.py`

Contains tolerant read logic for old row layouts.

Responsibilities:

- Repair old compact rows
- Repair shifted rows
- Map legacy headers to current fields
- Keep this logic isolated so it does not pollute normal save code

`save_service.py`

Coordinates a full save operation.

Responsibilities:

- Collect normalized values
- Validate
- Save audit entry
- Save robot-side fields if needed
- Create follow-up action if requested
- Trigger compatibility preview/update
- Emit save result

Acceptance Criteria

- Normalization can be tested without opening Excel.
 - Validation can be tested without opening Excel.
 - Completion can be tested without opening Excel.
 - Workbook read/write code is isolated.
 - Legacy repair code is isolated.
 - Existing public functions can remain as compatibility wrappers during transition.
-

Feature / Refactor 7: Versioned Workbook Migrations

Problem

The workbook schema evolves over time. Fields get added, renamed, reordered, and migrated.

Examples:

- `Number of Vacuum Cups` becoming `Number of Parts Picked`
- Gripper fields being added/changed
- Pneumatic circuit fields being added
- Cylinder fields being added
- Manual completion override metadata being added

If workbook updates are handled as scattered procedural checks, the workbook becomes harder to reason about.

Goal

Create a versioned workbook migration framework.

Proposed Structure

```
core/workbook_migrations/  
├─ __init__.py  
├─ runner.py  
├─ metadata.py  
├─ v001_initial.py  
├─ v002_add_gripper_fields.py  
├─ v003_rename_number_of_vacuum_cups.py  
├─ v004_add_pneumatic_circuits.py  
├─ v005_add_cylinders.py
```

```
|— v006_add_manual_completion_override.py
|— v007_future_change.py
```

Migration Metadata Storage

Recommended: hidden workbook sheet.

Sheet name:

```
_EOAT_Metadata
```

Fields:

```
Key
Value
Updated At
```

Example rows:

```
workbook_schema_version | 0.1.6 | 2026-05-28T20:15:00
migration:v005_add_cylinders | applied | 2026-05-28T20:15:00
migration:v006_add_manual_completion_override | applied | 2026-05-28T20:16:00
```

Alternative: store this in a project metadata JSON file.

```
00_Project_Admin/project_metadata.json
```

The hidden workbook sheet is better when the workbook may travel by itself.

Migration Runner Behavior

The runner should:

1. Open workbook.
2. Create backup.
3. Read current migration metadata.
4. Determine pending migrations.
5. Apply pending migrations in order.
6. Save workbook.
7. Log migration result.
8. Return a structured result.

Migration Object Pattern

```
@dataclass(frozen=True)
class WorkbookMigration:
    migration_id: str
    description: str
    apply: Callable[[Workbook], MigrationResult]
```

Example:

```
MIGRATION = WorkbookMigration(
    migration_id="v005_add_cylinders",
    description="Add cylinder count and cylinder type fields to EOAT
Inventory.",
    apply=apply_migration,
)
```

Cylinder Migration

Migration ID: v005_add_cylinders

Actions:

- Add # of Cylinders column if missing.
- Add Cylinder Type column if missing.
- Place columns near other tooling fields.
- Apply integer validation to # of Cylinders if workbook validation support exists.
- Apply dropdown validation to Cylinder Type: Linear, Rotary.
- Do not modify existing audit rows except to leave new fields blank.

Manual Override Migration

Migration ID: v006_add_manual_completion_override

Actions:

- Add Manual Completion Override column if missing.
- Add Manual Completion Override Timestamp column if missing.
- Add Ignored Empty Fields At Override column if missing.
- Optional: add Manual Completion Override Note column.
- Do not set override values for existing audits.

Acceptance Criteria

- Running migrations twice is safe.
 - Missing columns are added once.
 - Existing data is preserved.
 - Workbook backups are created before migration.
 - Migration history is recorded.
 - Migration result is visible in Workbook Health or logs.
-

Feature / Refactor 8: Audit Relationship Service for Press View and Compatibility

Problem

Press View, Compatibility Entry, Audit Progress, and reports all need to understand relationships between:

- Physical audit rows
- Compatibility-derived rows
- Source audit IDs
- Press/machine numbers
- Tool numbers
- Robot identities
- Compatibility source metadata

If each page computes these relationships independently, they can disagree.

This can cause issues like one page showing one compatible entry while another page shows several compatible entries.

That is not a UI problem. That is a shared relationship model problem.

Goal

Create one shared service responsible for resolving audit and compatibility relationships.

Proposed File

```
core/audit/relationships.py
```

Proposed API

```
@dataclass(frozen=True)
class AuditRelationshipSummary:
    machine: str
    physical_audits: tuple[dict, ...]
    compatibility_entries: tuple[dict, ...]
    source_audits: tuple[dict, ...]
    compatible_tools: tuple[str, ...]
    warnings: tuple[str, ...]

def physical_audits_for_machine(project_root: str, machine: str) -> list[dict]:
    ...

def compatibility_entries_for_machine(project_root: str, machine: str) ->
list[dict]:
    ...

def compatibility_entries_for_source_audit(project_root: str, audit_id: str) ->
list[dict]:
    ...

def source_audit_for_compatibility_row(project_root: str, row: dict) -> dict |
None:
    ...

def relationship_summary_for_machine(project_root: str, machine: str) ->
AuditRelationshipSummary:
    ...
```

Rules

Physical Audit Rows

A physical audit row is an actual observed EOAT audit.

It should usually have:

- Entry Type = Audited
- Audit ID

- Press/Machine #
- Audit Date
- Auditor

Compatibility Rows

A compatibility row is derived/generated from audit data.

It should usually have:

- Entry Type = Compatible
- Source Audit ID
- Compatibility Source
- Press/Machine #
- Tool #

Counting Rules

- Press View should show physical audits and compatible entries separately.
- Audit Progress should not count compatibility rows as physical audits.
- Compatibility counts should use the same resolver everywhere.
- Source audit relationships should be traceable.

Acceptance Criteria

- Press View and Compatibility Entry use the same service.
- Audit Progress uses the same service.
- Workbook Health uses the same service for relationship warnings.
- Reports use the same service for compatibility summaries.
- A machine with seven compatibility rows shows seven everywhere unless filters are intentionally applied and visibly explained.

Testing Requirements

Add tests for:

1. Physical audit rows are detected correctly.
 2. Compatibility rows are detected correctly.
 3. Compatibility rows link back to source audits correctly.
 4. Press View count matches Compatibility Entry count for the same machine/filter.
 5. Blank autofilled metadata fields are ignored when appropriate.
 6. Missing source audit IDs create warnings but do not crash.
-

Feature / Refactor 9: Dashboard Search / Navigation Route Registry

Problem

The dashboard currently has a centralized page registry, which is good. But search result actions and deep-link actions can become a long chain of `if action == ...` blocks.

That pattern becomes harder to maintain as more pages and searchable objects are added.

Goal

Create a route registry for opening search results and annotation targets.

Proposed File

```
app/search_routes.py
```

Proposed Data Model

```
from dataclasses import dataclass
from typing import Callable

@dataclass(frozen=True)
class SearchRoute:
    action: str
    page_key: str
    handler: Callable
```

Example Routes

```
SEARCH_ROUTES = {
    "open_audit": OpenAuditRoute(),
    "open_press": OpenPressRoute(),
    "open_note": OpenNoteRoute(),
    "open_open_item": OpenOpenItemRoute(),
    "open_validation": OpenValidationRoute(),
    "open_report": OpenReportRoute(),
```

```
    "open_photo": OpenPhotoRoute(),  
}
```

Route Handler Example

```
class OpenAuditRoute:  
    page_key = "audit"  
  
    def open(self, window, data: dict) -> bool:  
        window.navigate_to_page("audit")  
        page = window.page("audit")  
        audit_id = str(data.get("audit_id") or "")  
        if audit_id and hasattr(page, "load_existing_audit"):  
            page.load_existing_audit(audit_id, loaded_message=f"Opened from  
global search: {audit_id}")  
        field = str(data.get("field") or "")  
        if field and hasattr(page, "focus_annotation_target"):  
            page.focus_annotation_target({  
                "audit_id": audit_id,  
                "field_key": field,  
                "field_label": field,  
                "target_type": "audit_field",  
            })  
        return True
```

Benefits

- Search actions become modular.
- New pages can register their own open behavior.
- Dashboard code becomes smaller.
- Deep-link behavior becomes easier to test.
- Broken routes can fail gracefully.

Acceptance Criteria

- Existing search result actions still work.
- Dashboard no longer needs a long action-specific chain.
- Route handlers can be tested individually.
- Unknown actions fail gracefully.

Feature / Refactor 10: Event Bus Hardening

Problem

The app has a useful event bus for events like audit saved, workbook validated, report generated, project root changed, settings changed, and scheduled report ran.

However, event handlers should be isolated. If one page listener fails, it should not prevent other listeners from receiving the event.

Goal

Make event publishing safer and easier to debug.

Proposed Improvements

1. Wrap event handlers in try/except

```
def publish(self, event: AppEvent) -> None:
    handlers = self._handlers_for(event)
    for handler in handlers:
        try:
            handler(event)
        except Exception as exc:
            log_event_handler_error(event, handler, exc)
```

2. Add event logging in debug mode

Record:

- Event type
- Source
- Timestamp
- Handler name
- Success/failure
- Exception summary if failed

3. Add typed payload helpers later

Instead of every event carrying arbitrary dictionaries forever, important events can get helper constructors.

Example:

```
def emit_audit_saved(event_bus, audit_id: str, workbook_path: str,
compatibility_updated: bool) -> None:
    event_bus.emit(
        EVENT_AUDIT_SAVED,
        {
            "audit_id": audit_id,
            "workbook_path": workbook_path,
            "compatibility_updated": compatibility_updated,
        },
        source="audit_save",
    )
```

This is not full enterprise ceremony. It is just enough structure to stop payload dictionaries from becoming mystery soup.

Acceptance Criteria

- A failing event handler does not break other subscribers.
- Event handler failures are logged.
- Existing event behavior remains compatible.
- Tests cover multiple handlers where one raises an exception.

Feature / Refactor 11: Stable Field IDs Separate from Labels

Problem

User-facing labels change over time.

Examples:

- `Number of Vacuum Cups` was changed to `Number of Parts Picked`.
- `Gripper Type` behavior changed.
- Cylinder fields are being added.
- Future fields may be renamed for clarity.

If the app uses display labels as internal identity everywhere, renames become risky.

Goal

Use stable internal field IDs while preserving user-friendly labels and workbook headers.

Example

```
AuditFieldSpec(  
  field_id="number_of_parts_picked",  
  label="Number of Parts Picked",  
  workbook_header="Number of Parts Picked",  
  legacy_headers=("Number of Vacuum Cups",),  
)
```

For cylinders:

```
AuditFieldSpec(  
  field_id="cylinder_count",  
  label="# of Cylinders",  
  workbook_header="# of Cylinders",  
)  
  
AuditFieldSpec(  
  field_id="cylinder_type",  
  label="Cylinder Type",  
  workbook_header="Cylinder Type",  
)
```

Benefits

- Labels can be improved without breaking saved data logic.
- Workbook headers can be migrated safely.
- Tests can refer to stable IDs.
- Reports can still show user-friendly names.
- Legacy imports become easier.

Acceptance Criteria

- Every field has a stable `field_id`.
 - Field IDs are unique.
 - Workbook headers can differ from field IDs.
 - Legacy headers can map to current fields.
 - User-facing labels remain unchanged unless intentionally updated.
-

Feature / Refactor 12: Stronger Tooling, Tests, and Release Discipline

Problem

The app is growing quickly. As features expand, basic mistakes become more likely:

- unused imports
- accidental circular imports
- duplicated constants
- inconsistent field names
- broken tests after refactors
- missing migration updates
- UI behavior drifting from core behavior

The app already has tests, which is good. It should now add lightweight linting and stronger test categories.

Recommended Additions

Add `pyproject.toml`

```
[tool.ruff]
line-length = 120

[tool.ruff.lint]
select = ["E", "F", "I", "B", "UP", "SIM"]
ignore = []

[tool.pytest.ini_options]
testpaths = ["tests"]
```

Add development dependencies

```
ruff
pytest-qt
coverage
```

Optional:

```
mypy
pyright
```

Type checking can be added gradually. Do not let type-checking become a side quest that eats the internship.

Recommended Test Groups

```
tests/core/audit/test_schema.py
tests/core/audit/test_completion.py
tests/core/audit/test_validation.py
tests/core/audit/test_normalization.py
tests/core/audit/test_relationships.py
tests/core/workbook/test_migrations.py
tests/app/audit/test_audit_form_ui.py
tests/app/audit/test_manual_override_ui.py
tests/app/test_search_routes.py
tests/app/test_event_bus.py
```

Recommended CI / Local Commands

```
python -m pytest
python -m ruff check .
python scripts/repo_safety_audit.py --root .
```

Acceptance Criteria

- Linting catches unused imports and obvious mistakes.
- Tests are grouped by domain.
- Core audit logic can be tested without launching PySide.
- UI smoke tests cover only actual UI behavior.
- Safety audit remains part of release readiness.

Recommended Implementation Phases for Codex

Phase 1: Add audit field registry foundation

Goal

Create the central audit field schema without changing behavior.

Codex Tasks

1. Create `core/audit/schema.py`.

2. Define `AuditFieldSpec`.
3. Add all existing audit fields to the registry.
4. Add helper functions for fields by section, group, workbook header, and field ID.
5. Add tests for registry completeness.
6. Keep existing constants working for backward compatibility.

Do Not Do Yet

- Do not rewrite the whole Audit page.
- Do not change workbook save behavior.
- Do not add cylinders yet unless asked separately.

Phase 2: Extract completion policy engine

Goal

Move audit progress/completion rules into a testable core module.

Codex Tasks

1. Create `core/audit/completion.py`.
2. Define field and section completion status objects.
3. Implement normal field completion behavior.
4. Add excluded progress policy.
5. Add optional group policy.
6. Add manual override support.
7. Update audit coach to consume completion results.
8. Add tests.

Acceptance

Existing completion results should stay the same except where explicitly configured differently.

Phase 3: Implement Cylinder fields

Goal

Add cylinder tracking cleanly using the registry and completion policy.

Codex Tasks

1. Add `# of Cylinders` field spec.
2. Add `Cylinder Type` field spec.
3. Add Cylinder Details group on EOAT Type and Tooling tab.
4. Add workbook headers.
5. Add workbook migration.
6. Add validation.

7. Add completion policy for optional group.
8. Add UI and core tests.

Acceptance

- Section is always visible.
- Cylinder Type defaults to Linear.
- Empty cylinder group does not reduce progress.
- Filled cylinder count activates the group.
- Existing audits load correctly.

Phase 4: Implement Manual Completion Override

Goal

Allow audit-specific completion override with truthful metadata.

Codex Tasks

1. Add override metadata fields.
2. Add workbook migration.
3. Add manual override button.
4. Add confirmation popup.
5. Save override metadata.
6. Update completion engine.
7. Update audit coach display.
8. Add tests.

Acceptance

- Override affects only current audit.
- Completion shows 100% by override.
- Blank fields are not falsely marked verified.
- Override persists after reload.

Phase 5: Split Audit page safely

Goal

Reduce `AuditPage` size and responsibility.

Codex Tasks

Extract modules in this order:

1. `form_state.py`
2. `field_widgets.py`

3. `form_builder.py`
4. `draft_controller.py`
5. `visibility_controller.py`
6. `coach_controller.py`
7. `annotation_controller.py`
8. `machine_lookup_controller.py`
9. `compatibility_controller.py`

Acceptance

- Behavior remains unchanged.
- Tests pass after each extraction.
- AuditPage becomes mostly orchestration.

Phase 6: Separate workbook repository logic

Goal

Make audit domain logic independent from Excel persistence.

Codex Tasks

1. Create audit models.
2. Create normalizer module.
3. Create validator module.
4. Create workbook repository module.
5. Move legacy repair logic.
6. Keep old public functions as wrappers.
7. Add tests.

Acceptance

Core audit rules can be tested without opening workbooks.

Phase 7: Add relationship service

Goal

Make Press View, Compatibility Entry, Audit Progress, and reports agree about physical audits and compatibility rows.

Codex Tasks

1. Create `core/audit/relationships.py`.
2. Implement physical audit row resolver.
3. Implement compatibility row resolver.
4. Implement source audit resolver.

5. Update Press View to use service.
6. Update Audit Progress to use service.
7. Update Workbook Health relationship warnings.
8. Add tests.

Acceptance

Counts match across pages unless a visible filter explains the difference.

Phase 8: Dashboard route registry and event bus hardening

Goal

Clean up dashboard action routing and make events safer.

Codex Tasks

1. Create search route registry.
2. Move search action handling out of DashboardWindow.
3. Add event bus exception isolation.
4. Add event handler failure logging.
5. Add tests.

Codex Prompt: Full Architecture Improvement Implementation Plan

Use this prompt when you want Codex to begin implementing the restructuring plan.

I need you to improve the EOAT Command Center architecture without changing user-facing behavior unless explicitly specified.

Focus on maintainability, correctness, and future feature scalability. Do not focus on raw latency/performance tuning in this pass.

Important: do not include tag-category work in this task.

Primary goals:

1. Add a central audit field registry.
2. Extract audit completion/progress logic into a dedicated completion policy engine.
3. Prepare the app for optional audit field groups such as Cylinders.
4. Prepare the app for manual audit completion override metadata.

5. Gradually split the large Audit page into smaller focused controllers.
6. Separate workbook persistence from audit domain logic.
7. Add a versioned workbook migration framework.
8. Add a shared audit relationship service so Press View, Compatibility Entry, Audit Progress, Workbook Health, and reports agree about physical audits and compatibility rows.
9. Improve dashboard search/action routing with a route registry.
10. Harden the event bus so one failing listener does not break unrelated UI updates.
11. Add stable internal field IDs separate from user-facing labels.
12. Add/strengthen tests for all changed behavior.

Do this in small phases. Do not rewrite the entire app at once.

Phase 1: Audit field registry foundation

- Create core/audit/schema.py.
- Define AuditFieldSpec.
- Move existing audit field definitions into the registry.
- Include field_id, label, workbook_header, section, group, widget type, dropdown values, default, numeric flag, required/important flags, visibility rule, progress policy, optional group, storage target, and legacy headers.
- Add helper functions to get fields by ID, workbook header, section, group, and storage target.
- Add tests proving the registry covers all current EOAT Inventory headers.
- Keep existing constants and behavior working.

Phase 2: Completion policy engine

- Create core/audit/completion.py.
- Move audit completion/progress logic out of UI code.
- Support normal fields, excluded fields, optional groups, non-applicable fields, stale conflicts, Unknown / Not Checked, and manual override.
- The completion engine must return field-level and audit-level truth, not just a percentage.
- Update the audit coach to consume the completion engine.
- Add tests.

Phase 3: Cylinder support

- Add # of Cylinders and Cylinder Type to the registry.
- Add Cylinder Details mini-section to EOAT Type and Tooling.
- Cylinder section must always be visible.
- Cylinder Type dropdown values: Linear and Rotary.
- Cylinder Type default: Linear.
- If # of Cylinders is blank, the cylinder group should not count against progress, even if Cylinder Type is showing Linear by default.
- If # of Cylinders is filled, cylinder fields should count normally.
- Add workbook headers and migration.
- Add validation and tests.

Phase 4: Manual completion override

- Add a button labeled Manual Override: Mark Audit Complete.
- Show a confirmation popup before applying the override.
- Store audit-specific override metadata.
- Completion should show 100% by manual override without pretending all fields were verified.
- Override should persist after save/reload.
- Override should affect only the current audit.
- Add tests.

Phase 5: Split AuditPage

- Extract form state, field widget creation, form building, draft handling, visibility, coach coordination, annotation coordination, machine lookup, and compatibility coordination into focused modules.
- Preserve behavior after each extraction.
- Keep AuditPage as a thin orchestration shell.

Phase 6: Workbook persistence separation

- Create audit models, normalizer, validator, workbook repository, workbook migrations, and legacy repair modules.
- Keep existing public functions as wrappers while transitioning.
- Ensure core audit logic can be tested without opening Excel.

Phase 7: Relationship service

- Create core/audit/relationships.py.
- Provide shared functions for physical audits, compatibility rows, source audit links, and machine summaries.
- Update Press View, Compatibility Entry, Audit Progress, Workbook Health, and reports to use this service.
- Add tests to prevent count mismatches.

Phase 8: Dashboard routing and event bus hardening

- Move search/deep-link action handling into a route registry.
- Add event bus exception isolation and logging.
- Add tests.

General requirements:

- Keep the app local-first.
- Do not move real project data into the repository.
- Do not break demo project support.
- Do not remove existing workbook compatibility behavior.
- Do not make destructive workbook changes without backups.
- Add tests for each phase.
- Run pytest after each phase.
- Provide a summary of files changed, behavior preserved, new tests added, and any migration notes.

Final Recommendation

The most important change is the audit field registry. Everything else becomes easier after that.

The recommended order is:

1. Field registry
2. Completion policy engine
3. Cylinder support
4. Manual completion override
5. AuditPage split
6. Workbook repository/migrations
7. Relationship service
8. Dashboard routing/event bus hardening

This keeps the work manageable and avoids turning the app into a giant refactor bonfire.

The goal is not to make the code look fancy. The goal is to make future changes boring.

Boring is good. Boring means adding a field does not accidentally break Press View, Compatibility Entry, scheduled reports, and the emotional stability of everyone within 40 feet of the keyboard.